



UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE LOS VALLES
Lic. En Tecnologías de la Información.

CRUD Python - PyMySQL

Desarrollo de Base de Datos Web

Alumno: Sergio Alejandro Rodríguez Hermosillo.

Profesor: Adolfo Castillo Chavarín

Martes, 15 de Septiembre de 2026.

1. ¿Cuál es el propósito principal de pymysql.connect() en PyMySQL?

- a) Crear una nueva base de datos MySQL.
- b) Establecer una conexión desde Python a una base de datos MySQL/MariaDB.
- c) Ejecutar consultas SQL sin conexión.
- d) Generar reportes en PDF.

pymysql.connect() abre un socket de red TCP/IP entre el entorno de ejecución Python y el motor de MySQL, autenticándose con credenciales que definí en el script (host, user, password y la BD dbPython).



```
1  self.conexion = pymysql.connect(  
2      host=self.host,  
3      user=self.user,  
4      password=self.password,  
5      database=self.database  
6  )
```

2. ¿Antes de ejecutar comandos SQL con PyMySQL, ¿qué objeto se debe obtener de la conexión?

- a) Cursor
- b) Commit
- c) Execute
- d) Connector

La conexión administra el enlace con el servidor, mientras que el Cursor es el objeto que utilizo para enviar y ejecutar las instrucciones SQL hacia MySQL.



```
1  cursor.execute(  
2      """CREATE TABLE usuario1 (  
3          Idusuario INT PRIMARY KEY AUTO_INCREMENT,  
4          nombres VARCHAR(25),  
5          apellidoPaterno VARCHAR(25),  
6          apellidoMaterno VARCHAR(25),  
7          user VARCHAR(10),  
8          pwd VARCHAR(10)  
9      )"""
```

3. ¿Tras ejecutar una operación de INSERT, UPDATE o DELETE, ¿qué método de la conexión es necesario invocar para guardar los cambios en la base de datos?

- a) cursor.save()
- b) `conexion.commit()`
- c) `conexion.execute()`
- d) cursor.close()

PyMySQL maneja las operaciones de escritura de forma transaccional. Invoco el método `.commit()` para confirmar las modificaciones en mi sesión y guardarlas permanentemente en el servidor.



```
1 self.conexion.commit()
2 cursor.close()
```

4. ¿Qué método del objeto Cursor se utiliza para enviar una consulta SQL al servidor?

- a) run()
- b) commit()
- c) execute()
- d) open()

Utilizo .execute() para transmitir la instrucción SQL junto con los datos parametrizados hacia el motor de MySQL para su procesamiento seguro.



```
1  sql = "DELETE FROM usuario1 WHERE Idusuario = %s"
2      cursor.execute(sql, (idusuario,))
```

5. ¿Para recuperar todos los resultados de una consulta, en lugar de `cursor.fetchone()`, ¿qué método se emplea?

- a) `cursor.getall()`
- b) `cursor.queryall()`
- c) `cursor.fetchall()`
- d) `cursor.results()`

Uso `.fetchall()` para extraer la totalidad de las filas devueltas por una instrucción `SELECT` en forma de lista o tabla.



```
1  sql = "SELECT * FROM usuario1 WHERE Idusuario = %s"
2      cursor.execute(sql, (idusuario,))
3      filas = cursor.fetchall()
```

6. ¿Cómo se configuran los parámetros de conexión (host, usuario, contraseña, base de datos) en PyMySQL?

- a) Usando variables globales.
- b) Pasándolos como argumentos a `pymysql.connect()`.
- c) En un archivo de configuración externo.
- d) Directamente en `cursor.execute()`.

Configuro los datos de conexión mediante argumentos nombrados (keyword arguments), utilizando las constantes que defini al inicio de mi script.



```
1  # Defino las variables globales para conectarme a MySQL en XAMPP
2  DB_HOST = 'localhost'
3  DB_USER = 'root'
4  DB_PASSWORD = '15054505'
5  DB_NAME = 'dbPython' # <-- Nombre explícito de mi base de datos
```

7. ¿Si ocurre un error en la conexión o en la ejecución de una consulta, PyMySQL lanza:

- a) Una alerta en consola.
- b) Un objeto `pymysql.MySQLError`.
- c) Un objeto `MySQLWarning`.
- d) Un valor `None`.

`pymysql.MySQLError` es la clase base para las excepciones del conector. La captura dentro de bloques `try/except` en mis clases para evitar que el programa falle si ocurre algún problema con SQL o la conexión.



```
1  try:
2      cursor = self.conexion.cursor()
3      sql = "DELETE FROM usuario1 WHERE Idusuario = %s"
4      cursor.execute(sql, (idusuario,))
5      self.conexion.commit()
6      cursor.close()
7      print(f"<-- [ÉXITO] Usuario ID {idusuario} eliminado correctamente.")
8  except pymysql.MySQLError as err:
9      print(f"<-- [ERROR] No pude eliminar el usuario: {err}")
```

8. ¿Cuál de las siguientes no es una buena práctica al usar PyMySQL?

- a) Cerrar cursor y conexión al finalizar.
- b) Usar execute() con placeholders (%s) para evitar inyección SQL.
- c) Capturar excepciones específicas (OperationalError, IntegrityError).
- d) Construir cadenas SQL concatenando directamente la entrada del usuario.

Concatenar cadenas con la entrada del usuario genera vulnerabilidades de Inyección SQL. En mi script evito este problema aplicando parametrización con marcadores \$s.



```
1  sql = """UPDATE usuario1
2          SET nombres=%s, apellidoPaterno=%s, apellidoMaterno=%s, user=%s, pwd=%s
3          WHERE Idusuario=%s"""
```

9. ¿Si no se cierra explícitamente cursor o conexión, ¿qué riesgo existe?

- a) Se convierte la base de datos a SQLite automáticamente.
- b) Pueden quedar conexiones abiertas en el servidor.
- c) Se pierde la sintaxis SQL.
- d) No hay ningún efecto.

Dejar recursos abiertos consume sockets de red y memoria en el servidor MySQL. En mi código cierro los cursores con `cursor.close()` al terminar cada operación y cierro la conexión principal mediante `db.cerrar()`.



```
1  self.conexion.commit()
2  cursor.close()
```

10. ¿Qué ventaja principal ofrece PyMySQL frente a otros conectores en Python?

- a) Es 100% Python, sin dependencias compiladas.
- b) Puede conectarse a cualquier tipo de base de datos.
- c) Traduce automáticamente consultas a otros dialectos SQL.
- d) Genera interfaces gráficas para consultas.

Al estar desarrollado íntegramente en Python puro, puedo instalar e importar PyMySQL en mi proyecto sin necesidad de instalar librerías nativas en C ni dependencias compiladas del sistema.



```
1  # =====
2  # 0. IMPORTACIÓN DE LIBRERÍAS
3  import pymysql
4  # =====
5
```